A Polyhedral Approach to the Asymmetric Traveling Salesman Problem

Matteo Fischetti; Paolo Toth

Management Science, Vol. 43, No. 11. (Nov., 1997), pp. 1520-1536.

Stable URL:
http://links.jstor.org/sici?sici=0025-1909%28199711%2943%3A11%3C1520%3AAPATTA%3E2.0.CO%3B2-N

Management Science is currently published by INFORMS.

Your use of the JSTOR archive indicates your acceptance of JSTOR's Terms and Conditions of Use, available at http://www.jstor.org/about/terms.html. JSTOR's Terms and Conditions of Use provides, in part, that unless you have obtained prior permission, you may not download an entire issue of a journal or multiple copies of articles, and you may use content in the JSTOR archive only for your personal, non-commercial use.

Please contact the publisher regarding any further use of this work. Publisher contact information may be obtained at http://www.jstor.org/journals/informs.html.

Each copy of any part of a JSTOR transmission must contain the same copyright notice that appears on the screen or printed page of such transmission.

JSTOR is an independent not-for-profit organization dedicated to creating and preserving a digital archive of scholarly journals. For more information regarding JSTOR, please contact support@jstor.org.

http://www.jstor.org/
Thu Apr 13 09:20:36 2006

A Polyhedral Approach to the Asymmetric Traveling Salesman Problem

Matteo Fischetti • Paolo Toth
DMI, University of Udine, Udine, Italy
DEIS, University of Bologna, Bologna, Italy

Several branch-and-bound algorithms for the exact solution of the asymmetric traveling salesman problem (ATSP), based on the assignment problem (AP) relaxation, have been proposed in the literature. These algorithms perform very well for some instances (e.g., those with uniformly random integer costs), but very poorly for others. The aim of this paper is to evaluate the effectiveness of a branch-and-cut algorithm exploiting ATSP-specific facet-defining cuts, to be used to attack hard instances that cannot be solved by the AP-based procedures from the literature. We present new separation algorithms for some classes of facet-defining cuts, and a new variable-pricing technique for dealing with highly degenerate primal LP problems. A branch-and-cut algorithm based on these new results is designed and evaluated through computational analysis on several classes of both random and real-world instances. The outcome of the research is that, on hard instances, the branch-and-cut algorithm clearly outperforms the best AP-based algorithms from the literature.
(*Traveling Salesman Problem; Branch-and-cut Algorithms; Separation; Computational Analysis*)

1. Introduction

The *Traveling Salesman Problem* (TSP) is one of the best-known combinatorial optimization problems. In this paper we present an exact algorithm for the asymmetric version of the problem, called ATSP in the sequel.

ATSP can formally be defined as follows. We are given a complete (loop-free) digraph $G = (V, A)$, where $V := \{1, \ldots, n\}$ and $A := \{(i, j) : i, j \in V, i \neq j\}$ are the nodeset and the arcset of G, respectively. Each arc (i, j) has an associated cost c_{ij}, assumed without loss of generality nonnegative and integer. The problem consists of finding a Hamiltonian circuit (*tour*) whose total cost is minimized. ATSP is strongly NP-hard, and finds several practical applications in routing and scheduling problems.

A number of exact branch-and-bound algorithms have been proposed for ATSP, which are based on the well-known *Assignment Problem* (AP) relaxation of ATSP; see Balas and Toth (1985) and Laporte (1992) for a survey. These algorithms perform very well on those instances for which AP turns out to give a tight approximation of ATSP. Among these "easy" instances, the most commonly used as a computational benchmark are those in which costs are generated as uniformly random integers in a given interval. According to Carpaneto et al. (1995), however, there are other classes of more realistic (sometimes real-world) ATSP instances for which the performance of an AP-based code is quite satisfactory. These instances arise, e.g., in ATSP reformulation of flow-shop scheduling and in stacker crane problems.

On the other hand, several other classes of instances are known that cannot be solved to optimality by known AP-based algorithms from the literature. In a recent paper, we considered several such hard classes, and showed that the performance of an AP-based branch-and-bound code can be significantly improved by means of more sophisticated bounding procedures (Fischetti and Toth 1992). Nevertheless, in some cases, even this approach is not satisfactory. A remarkable example is given by Balas et al. (1993). These authors report computational experience with a 43-node real-

0025-1909/97/4311/1520$05.00
Copyright © 1997, Institute for Operations Research and the Management Sciences

world instance (called P43 in the sequel) that could not be solved to optimality by our code (Fischetti and Toth 1992), even allowing for 32 CPU hours of computing time on a SUN SPARCstation 330. By using a general-purpose branch-and-cut algorithm with an ad-hoc initialization, however, Balas et al. were able to solve P43 in a few minutes on the same computer. More recently, Jünger et al. (1995) transformed P43 into a symmetric TSP instance with 86 nodes, and solved it by means of their branch-and-cut code for the symmetric TSP. These results suggest that an ATSP instance that is very hard to solve by means of an AP-based code can in some cases be solved rather easily through a branch-and-cut scheme. To our knowledge, however, no computational testing on the performance of a branch-and-cut ATSP code has been reported so far, with a comparison with other techniques from the literature.

The aim of this paper is to design a specialized branch-and-cut algorithm based on facet-defining ATSP cuts, which can be successful in solving *hard* instances, i.e., instances that cannot be solved by AP-based codes from the literature. This is also motivated by a number of recent results on the polyhedral structure of ATSP, including those published in Balas (1989), Balas and Fischetti (1992, 1993a, 1993b), Chopra and Rinaldi (1990), Fischetti (1991, 1995), and Queyranne and Wang (1995).

We present new separation algorithms for some classes of facet-defining cuts, discuss node shrinking as a separation tool, and describe a new variable-pricing technique intended for highly degenerate primal LP problems. A branch-and-cut algorithm based on these new results is designed and evaluated through computational analysis on several classes of both random and real-world hard instances. The outcome of the research is that the branch-and-cut algorithm clearly outperforms (on hard instances) the best AP-based algorithm from the literature. Moreover, the use of ATSP-specific cuts (other than subtour-elimination constraints) is of fundamental importance for the solution of some real-world instance.

The paper is organized as follows. In §2, we give a basic ILP model for ATSP and describe separation algorithms for symmetric inequalities (§2.1) and for some main classes of ATSP-specific cuts, namely D_k^+ and D_k^- inequalities (§2.2), odd CAT inequalities (§2.3), and clique-lifted inequalities (§2.4). In §3, we discuss the new variable-pricing scheme. Our overall branch-and-cut algorithm is outlined in §4. Computational results are presented in §5. Some conclusions are drawn in §6.

2. Facet-Defining Inequalities and Separation

Here is a standard ILP model for ATSP. For each arc $(i, j) \in A$, let $x_{ij} = 1$ if the arc is selected in the optimal tour; $x_{ij} = 0$ otherwise. Given two nodesets S_1 and S_2, we write $x(S_1, S_2)$ for $\Sigma\{x_{ij} : (i, j) \in A, i \in S_1, j \in S_2\}$. To simplify notation, singleton sets like $\{v\}$ are sometimes denoted by v. ATSP then calls for

$$v(\text{ATSP}) := \min \sum_{(i,j)\in A} c_{ij}x_{ij}, \tag{1}$$

subject to

$$x(i, V) = 1 \quad \text{for } i \in V, \tag{2}$$

$$x(V, j) = 1 \quad \text{for } j \in V, \tag{3}$$

$$x(S, S) \leq |S| - 1 \quad \text{for } S \subset V, 2 \leq |S| \leq n - 2, \tag{4}$$

$$x_{ij} \geq 0 \quad \text{for } (i, j) \in A, \tag{5}$$

$$x_{ij} \text{ integer} \quad \text{for } (i, j) \in A. \tag{6}$$

Constraints (2) and (3) are called *out-* and *in-degree equations*, respectively, whereas (4) are known as *Subtour Elimination Constraints* (SECs, for short). Removing SECs (and, consequently, the integrality requirement) yields the so-called *Assignment Problem* (AP), which can be solved efficiently by means of specialized $O(n^3)$-time algorithms (see, e.g., Carpaneto and Toth 1987).

The linear model (2)–(5) has a number of redundant constraints. In particular, one of the equations (2)–(3) can be omitted. Moreover, all SEC pairs $x(S, S) \leq |S| - 1$ and $x(V\backslash S, V\backslash S) \leq |V\backslash S| - 1$ are equivalent. When the digraph G is complete, no other redundancy exists.

We are interested in the ATSP polytope

$$P := \text{conv}\{x \in \mathbb{R}^A : (2.2) - (2.6) \text{ hold}\},$$

whose vertices are the characteristic vectors of the tours of G. Grötschel and Padberg (1985) proved that P has dimension $\dim(P) = |A| - (2n - 1)$, i.e., every valid equation for P is a linear combination of the degree equations (2)–(3). Moreover, they showed that all SECs

(4) define facets of P; any two such facets are distinct, unless defined by two sets S_1 and S_2 with $S_2 = V \backslash S_1$.

We next consider additional classes of facet-inducing inequalities for P that we exploit within our branch-and-cut algorithm. These classes proved to be of crucial importance for the solution of some real-world instances. For each class, we address the associated *separation* problem (in its optimization version), defined as follows: Given a point $x^* \geq 0$ satisfying the degree equations, and a family $\mathcal{F}$ of ATSP inequalities, find a most violated member of $\mathcal{F}$, i.e., an inequality $\alpha x \leq \alpha_0$ belonging to $\mathcal{F}$ and maximizing the *degree of violation* $\alpha x^* - \alpha_0$.

2.1. Symmetric Inequalities

An ATSP inequality $\alpha x \leq \alpha_0$ is called *symmetric* when $\alpha_{ij} = \alpha_{ji}$ for all $(i, j) \in A$. Symmetric inequalities can be thought of as derived from valid inequalities for the Symmetric Traveling Salesman Problem (STSP), defined as the problem of finding a minimum-cost Hamiltonian cycle in a given undirected graph $G_E = (V, E)$. Let $y_e = 1$ if edge $e \in E$ belongs to the optimal STSP solution; $y_e = 0$ otherwise. Every inequality $\Sigma_{e \in E} \alpha_e y_e \leq \alpha_0$ for STSP can be transformed into a valid ATSP inequality by simply replacing y_e by $x_{ij} + x_{ji}$ for all $e = [i, j] \in E$. This produces the symmetric inequality $\alpha x \leq \alpha_0$, where $\alpha_{ij} = \alpha_{ji} = \alpha_{[i,j]}$ for all $i, j \in V$, $i \neq j$. Conversely, every symmetric ATSP inequality $\alpha x \leq \alpha_0$ corresponds to the valid STSP inequality $\Sigma_{[i,j] \in E} \alpha_{ij} y_{[i,j]} \leq \alpha_0$.

The above correspondence implies that every separation algorithm for STSP can be used, as a "black box", for ATSP as well. To this end, given the point x^* one first defines the undirected counterpart y^* of x^* by means of the transformation

$$y_e^* := x_{ij}^* + x_{ji}^* \quad \text{for all } e = [i, j] \in E,$$

and then applies the STSP separation algorithm to y^*. On return, the detected most violated STSP inequality is transformed into its ATSP counterpart, both inequalities having the same degree of violation.

Several exact/heuristic separation algorithms for STSP have been proposed in recent years, all of which can be used for ATSP. In our computational study, we used only two such separation tools, namely:

(i) the Padberg-Rinaldi (1990a) exact algorithm for SECs; and

(ii) the simplest heuristic scheme for comb (actually, 2-matching) constraints, in which the components of the graph induced by the edges $e \in E$ with fractional y_e^* are considered as potential handles of the comb.

Clearly, more sophisticated STSP separation procedures could have been used, see e.g., Padberg and Rinaldi (1990b), Jünger et al. (1995), Applegate et al. (1995), and Clochard and Naddef (1993). Since we are mainly concerned with the use of ATSP-specific cuts in this paper, we decided to leave this issue to future work.

2.2. D_k^+ and D_k^- Inequalities

The following D_k^+ *inequalities* have been proposed by Grötschel and Padberg (1985):

$$x_{i_1 i_k} + \sum_{h=2}^{k} x_{i_h i_{h-1}} + 2 \sum_{h=2}^{k-1} x_{i_1 i_h} + \sum_{h=3}^{k-1} x(\{i_2, \ldots, i_{h-1}\}, i_h) \leq k - 1, \tag{7}$$

where $(i_1, \ldots, i_k)$ is any sequence of $k \in \{3, \ldots, n-1\}$ distinct nodes. D_k^+ inequalities are facet-inducing for P, as proved by Fischetti (1991), and are obtained by lifting the cycle inequality $\Sigma_{(i,j) \in C} x_{ij} \leq k - 1$ associated with the circuit $C := \{(i_1, i_k), (i_k, i_{k-1}), \ldots, (i_2, i_1)\}$. Notice that the node indexes along C are different from those used in the original Grötschel-Padberg (1985) definition; our notation allows for a simplified description of the forthcoming separation procedure.

As a slight extension of the original definition, we allow for $k = 1, 2$ in the sequel, in which cases (7) degenerates into the valid constraints $x_{i_1 i_1} \leq 0$ and $x_{i_1 i_2} + x_{i_2 i_1} \leq 1$, respectively.

The separation problem for the class of D_k^+ inequalities calls for a node sequence $(i_1, \ldots, i_k)$, $1 \leq k \leq n - 1$, for which the degree of violation

$$\phi(i_1, \ldots, i_k) := x_{i_1 i_k}^* + \sum_{h=2}^{k} x_{i_h i_{h-1}}^* + 2 \sum_{h=2}^{k-1} x_{i_1 i_h}^* + \sum_{h=3}^{k-1} x^*(\{i_2, \ldots, i_{h-1}\}, i_h) - k + 1 \tag{8}$$

is as large as possible. This is itself a combinatorial optimization problem that can be solved by the following implicit enumeration scheme.

We start with an empty node sequence. Then, iteratively, we extend the current sequence in any possible way and evaluate the degree of violation of the corresponding D_k^+ inequality. The process can be visualized by means of a branch-decision tree. The root node (level 0) of the tree represents the empty sequence. Each node at level k ($1 \le k \le n-1$) corresponds to a sequence of the type $(i_1, \ldots, i_k)$; when $k \le n-2$, each such node generates $n-k$ son nodes, one for each possible extended sequence $(i_1, \ldots, i_k, i_{k+1})$. Exhaustive enumeration of all nodes of the tree is clearly impractical, even for small values of n. On the other hand, a very large number of these nodes can be pruned (along with the associated subtrees) by means of the following upper bound computation. Let $(i_1, \ldots, i_k)$, $1 \le k \le n-2$, be the sequence associated with the current branching node, say μ, and let $\phi_{\max}$ denote the maximum degree of violation so far found during the enumeration. Consider any potential descendent node of μ associated with a sequence of the type $(i_1, \ldots, i_k, i_{k+1}, \ldots, i_m)$. Then, directly from definition (8), one has

$$\begin{aligned}\phi(i_1, \ldots, i_k, i_{k+1}, \ldots, i_m) &\\ \le \pi(i_1, \ldots, i_k) + x^*_{i_{k+1}i_k} &+ [x^*(i_1, V) - 1]\\ + \sum_{h=k+1}^{m-1} &[x^*(V, i_h) - 1]\\ = \pi(i_1, \ldots, i_k) + x^*_{i_{k+1}i_k}, &\end{aligned} \tag{9}$$

where we have defined

$$\pi(i_1, \ldots, i_k) := \sum_{h=2}^{k} x^*_{i_h i_{h-1}} + \sum_{h=2}^{k} x^*(\{i_1, \ldots, i_{h-1}\}, i_h) - k + 1.$$

Observe that $\pi(i_1, \ldots, i_k)$ cannot exceed the degree of violation of the SEC associated with the nodeset $S := \{i_1, \ldots, i_k\}$; hence one has $\pi(i_1, \ldots, i_k) \le 0$ whenever all SECs are satisfied by x^*.

According to (9), the only son nodes of μ that need to be generated are those associated with a sequence $(i_1, \ldots, i_k, i_{k+1})$ such that

$$x^*_{i_k i_{k+1}} > \phi_{\max} - \pi(i_1, \ldots, i_k). \tag{10}$$

Notice that both quantities $\phi(i_1, \ldots, i_k)$ and $\pi(i_1, \ldots, i_k)$ can parametrically be computed along the branching tree as:

$$\phi(i_1, \ldots, i_k) = \phi(i_1, \ldots, i_{k-1}) + x^*_{i_1 i_k} + x^*_{i_k i_{k-1}} + x^*(\{i_1, \ldots, i_{k-2}\}, i_{k-1}) - 1,$$

and

$$\pi(i_1, \ldots, i_k) = \pi(i_1, \ldots, i_{k-1}) + x^*_{i_k i_{k-1}} + x^*(\{i_1, \ldots, i_{k-1}\}, i_k) - 1,$$

where $\phi(i_1) := \pi(i_1) := 0$ for all singleton sequences (i_1).

Restriction (10) is very effective in practice, and dramatically reduces the number of nodes typically generated in the enumeration. Nevertheless, in some cases one may be interested in further reducing the computing time spent in the procedure. To this end, before running the above-described exact enumeration we try a "truncated" version in which each node at level $k \ge 2$ generates at most one son node, namely the one associated with the sequence $(i_1, \ldots, i_k, i_{k+1})$, if any, where i_{k+1} satisfies $x^*_{i_{k+1}i_k} > 0$ and $x^*_{i_1 i_{k+1}} + x^*_{i_{k+1}i_k}$ is as large as possible.

The proposed separation scheme proved to be rather successful in that it requires a small fraction of the overall compution time and produces a considerable number of violated cuts; see §5 for details.

A final comment is in order. The performance of the overall branch-and-cut algorithm is generally improved if one generates, at each round of separation, a number of violated cuts (rather than the most violated one) for each family. In our implementation, we decided to generate, for each $i_1 \in V$, the most violated D_k^+ inequality associated with a node sequence starting with i_1. This is obtained by searching the decision tree in a depth-first manner, and resetting to zero the value $\phi_{\max}$ of the incumbent best sequence whenever we backtrack to a node at level 1.

We conclude this section by addressing the following D_k^- *inequalities*:

$$x_{i_k i_1} + \sum_{h=2}^{k} x_{i_{h-1}i_h} + 2\sum_{h=2}^{k-1} x_{i_h i_1} + \sum_{h=3}^{k-1} x(i_h, \{i_2, \ldots, i_{h-1}\}) \le k - 1, \tag{11}$$

where $(i_1, \ldots, i_k)$ is any sequence of $k \in \{3, \ldots, n-1\}$

distinct nodes. D_k^- inequalities are valid (Grötschel and Padberg 1985) and facet-inducing (Fischetti 1991) for P; they can be obtained by lifting the cycle inequality $\Sigma_{(i,j)\in C} x_{ij} \leq k - 1$ associated with the circuit $C := \{(i_1, i_2), \ldots, (i_{k-1}, i_k), (i_k, i_1)\}$.

D_k^- inequalities can be thought of as derived from D_k^+ inequalities by swapping the coefficient of the two arcs (i, j) and (j, i) for all $i, j \in V, i < j$. This is a perfectly general operation, called *transposition* in the sequel, that works as follows.

For every $\alpha \in \mathbb{R}^A$, let $\alpha^T \in \mathbb{R}^A$ be defined by: $\alpha_{ij}^T := \alpha_{ji}$ for all $(i, j) \in A$. Clearly, the inequality $\alpha x \leq \alpha_0$ is valid (or facet-inducing) for P if and only if its transposed version, $\alpha^T x \leq \alpha_0$, is. This follows from the obvious fact that $\alpha^T x = \alpha x^T$, where $x \in P$ if and only if $x^T \in P$. Moreover, every separation procedure for $\alpha x \leq \alpha_0$ can also be used, as a "black box," to deal with $\alpha^T x \leq \alpha_0$. To this end, one gives the transposed point $(x^*)^T$ (instead of x^*) on input to the procedure, and then transposes the returned inequality.

The above considerations show that both the heuristic and exact separation algorithms we have designed for D_k^+ inequalities can also be used for D_k^- inequalities.

2.3. Odd CAT Inequalities

The following class of inequalities has been proposed by Balas (1989). Two distinct arcs (i, j) and (u, v) are called *incompatible* if $i = u$, or $j = v$, or $i = v$ and $j = u$; *compatible* otherwise. A *Closed Alternating Trail* (CAT, for short) is a sequence $T = \{a_1, \ldots, a_t\}$ of t distinct arcs such that, for $k = 1, \ldots, t$, the arc a_k is incompatible with arcs a_{k-1} and a_{k+1}, and compatible with all other arcs in T (with $a_0 := a_t$ and $a_{t+1} := a_1$). Let $\delta^+(v)$ and $\delta^-(v)$ contain all the arcs of G leaving and entering node $v \in V$, respectively. Given a CAT T, a node v is called a *source* if $|\delta^+(v) \cap T| = 2$, whereas it is called a *sink* if $|\delta^-(v) \cap T| = 2$. Notice that a node can play both source and sink roles. Let Q be the set of the arcs $(i, j) \in A \backslash T$ such that i is a source and j is a sink node. For any CAT of odd length t, the following *odd CAT inequality*

$$\sum_{(i,j)\in T\cup Q} x_{ij} \leq \frac{|T| - 1}{2}, \tag{12}$$

is valid and facet-defining (except in two pathological cases arising for $n \leq 6$) for P.

We next describe a heuristic separation algorithm for the family of odd CAT inequalities. This algorithm is based on the known fact that odd CAT inequalities correspond to odd cycles on an auxiliary "incompatibility" graph (Balas 1989). Also, our separation algorithm can be viewed as a specialized version of a scheme proposed by Caprara and Fischetti (1996) for the separation of a subclass of Chvátal-Gomory cuts for general integer programming problems.

Given the point x^*, we set up an edge-weighted undirected graph $\tilde{G} = (\tilde{N}, \tilde{E})$ having a node ν_a for each arc $a \in A$ with $x_a^* > 0$, and an edge $e = [\nu_a, \nu_b]$ for each pair a, b of incompatible arcs, whose weight is defined as $w_e := 1 - (x_a^* + x_b^*)$. We assume that x^* satisfies all degree equations as well as all trivial SECs of the form $x_{ij} + x_{ji} \leq 1$; this implies $w_e \geq 0$ for all $e \in \tilde{E}$.

Let $\tilde{\delta}(v)$ contain the edges in $\tilde{E}$ incident with a given node $v \in \tilde{N}$. A *cycle* $\tilde{C}$ of $\tilde{G}$ is an edge subset of $\tilde{E}$ inducing a connected subgraph of $\tilde{G}$, and such that $|\tilde{C} \cap \tilde{\delta}(v)|$ is even for all $v \in \tilde{N}$. Cycle $\tilde{C}$ is called (i) *odd* if $|\tilde{C}|$ is odd; (ii) *simple* if $|\tilde{C} \cap \tilde{\delta}(v)| \in \{0, 2\}$ for all $v \in \tilde{N}$; and (iii) *chordless* if the subgraph of $\tilde{G}$ induced by the nodes covered by $\tilde{C}$ has no other edges than those in $\tilde{C}$.

By construction, every simple and chordless odd cycle $\tilde{C}$ in $\tilde{G}$ corresponds to an odd CAT T, where $a \in T$ if and only if ν_a is covered by $\tilde{C}$. In addition, the total weight of $\tilde{C}$ is

$$\begin{aligned} w(\tilde{C}) := \sum_{e\in\tilde{C}} w_e &= \sum_{[\nu_a,\nu_b]\in\tilde{C}} (1 - x_a^* - x_b^*) \\ &= |T| - 2 \sum_{a\in T} x_a^*, \end{aligned}$$

hence $(1 - w(\tilde{C}))/2$ gives a lower bound on the degree of violation of the corresponding CAT inequality, computed as

$$\phi(T) := \left(2 \sum_{a\in T\cup Q} x_a^* - |T| + 1\right) \Big/ 2.$$

The heuristic separation algorithm we propose computes, for each $e \in \tilde{E}$, a minimum-weight odd cycle $\tilde{C}_e$ that uses edge e. If $\tilde{C}_e$ happens to be simple and chordless, then it corresponds to an odd CAT, say T. If, in addition, the lower bound $(1 - w(\tilde{C}_e))/2$ exceeds a given threshold $\theta = -1/2$, then the corresponding inequality is hopefully violated; hence, we evaluate its actual degree of violation, $\phi(T)$, and store the inequality

if $\phi(T) > 0$. To avoid detecting twice the same inequality, we remove the edge e from $\tilde{G}$ after the computation of each $\tilde{C}_e$.

To increase the chances of finding odd cycles that are simple and chordless, we make all edge weights strictly positive by adding to them a small positive value ($\epsilon := 0.001$ in our implementation). This guarantees that ties are broken in favor of inclusion-minimal sets $\tilde{C}_e$. Notice, however, that a generic minimum-weight odd cycle $\tilde{C}_e$ need not be simple nor chordless even in this case, due to the fact that we impose $e \in \tilde{C}_e$. For example, $\tilde{C}_e$ may decompose into two simple cycles, say $\tilde{C}_e^1$ and $\tilde{C}_e^2$, where $\tilde{C}_e^1$ is of even cardinality and goes through edge e, and $\tilde{C}_e^2$ is of odd cardinality and overlaps $\tilde{C}_e^1$ in a node.

The key point of the algorithm is the computation in $\tilde{G}$ of a minimum-weight odd cycle going through a given edge. Assuming that the edge weights are all nonnegative, this problem is known to be polynomially solvable as it can be transformed into a shortest path problem; see Gerards and Schrijver (1986). To this end, one constructs an auxiliary bipartite undirected graph $G_B = (N_B' \cup N_B'', E_B)$ obtained from $\tilde{G}$ as follows. For each node ν in $\tilde{G}$ there are two nodes in G_B, say ν' and ν''. For each edge $e = [\nu_1, \nu_2]$ of $\tilde{G}$ there are two edges in G_B, namely $[\nu_1', \nu_2'']$ and $[\nu_2', \nu_1'']$, both having weight w_e. By construction, every minimum-weight odd cycle $\tilde{C}_e$ of $\tilde{G}$ going through edge $e = [\nu_1, \nu_2]$ corresponds in G_B to a shortest path from ν_1' to ν_2', plus the edge $[\nu_2', \nu_1'']$. Hence the computation of all $\tilde{C}_e$'s can be performed efficiently by computing for each ν_1' the shortest path from ν_1' to all other nodes in N_B'.

2.4. Clique Lifting and Shrinking

Clique lifting can be described as follows (see Balas and Fischetti 1993a for details): Let $P(G')$ denote the ATSP polytope associated with a given complete digraph $G' = (V', A')$. Given a valid inequality $\beta y \leq \beta_0$ for $P(G')$, we define

$$\beta_{hh} := \max\{\beta_{ih} + \beta_{hj} - \beta_{ij} : i, j \in V' \backslash \{h\}, i \neq j\} \quad \text{for all } h \in V',$$

and construct an enlarged complete digraph $G = (V, A)$ obtained from G' by replacing each node $h \in V'$ by a clique S_h containing at least one node (hence, $|V| = \Sigma_{h \in V'} |S_h| \geq |V'|$). In other words, $(S_1, \ldots, S_{|V'|})$ is a proper partition of V in which the hth set corresponds to the hth node in V'.

For all $v \in V$, let $v \in S_{h(v)}$. We define a new *clique lifted* inequality for $P(G)$, say $\alpha x \leq \alpha_0$, where $\alpha_0 := \beta_0 + \Sigma_{h \in V'} \beta_{hh}(|S_h| - 1)$ and $\alpha_{ij} := \beta_{h(i)h(j)}$ for each $(i, j) \in A$. It is shown in Balas and Fischetti (1993a) that the new inequality is always valid for $P(G)$; in addition, if the starting inequality $\beta x \leq \beta_0$ defines a facet of $P(G')$, then $\alpha x \leq \alpha_0$ is guaranteed to be facet-inducing for $P(G)$.

Clique lifting is a powerful theoretical tool for extending known classes of inequalities. Also, it has important applications in the design of separation algorithms in that it allows one to simplify the separation problem through the following *shrinking* procedure.

Let $S \subset V, 2 \leq |S| \leq n - 2$, be a node subset saturated by x^*, in the sense that $x^*(S, S) = |S| - 1$, and suppose S is shrunk into a single node, say σ, and x^* is updated accordingly. Let $G' = (V', A')$ denote the shrunken digraph, where $V' := (V \backslash S) \cup \{\sigma\}$, and let y^* be the shrunken counterpart of x^*. Every valid inequality $\beta y \leq \beta_0$ for $P(G')$ that is violated by y^* corresponds in G to an equally violated inequality, say $\alpha x \leq \alpha_0$, obtained through clique lifting by replacing back σ with the original set S. As observed by Padberg and Rinaldi (1990b), however, this shrinking operation can affect the possibility of detecting violated cuts on G', as it may produce a point y^* belonging to $P(G')$ even when $x^* \notin P(G)$.

For instance, let $n = 4$ and $x_{ij}^* = 1/2$ if $(i, j) \in \{(1, 2), (1, 4), (2, 1), (2, 3), (3, 1), (3, 4), (4, 2), (4, 3)\}$; $x_{ij}^* = 0$ otherwise. One readily checks that $x^* \notin P(G)$, as x^* violates, e.g., the D_3^+ inequality $x_{12} + x_{23} + x_{31} + 2x_{21} \leq 2$. On the other hand, shrinking the saturated set $S := \{1, 2\}$ produces a digraph G' with nodeset $V' := \{\sigma, 3, 4\}$, and a point y^* with $y_{ij}^* = 1/2$ for all $(i, j) \in A'$. But then y^* is the convex combination of the characteristic vectors of the two tours $(\sigma, 3, 4)$ and $(\sigma, 4, 3)$, hence y^* cannot be cut off by any linear cut as it belongs to $P(G')$.

The above example shows that shrinking has to be applied with some care. There are, however, simple conditions on the choice of S that guarantee $y^* \notin P(G')$, provided $x^* \notin P(G)$ as in the cases of interest for separation.

The simplest such condition concerns the shrinking of *1-arcs* (i.e., arcs (i, j) with $x_{ij}^* = 1$), and requires $S = \{i, j\}$ for a certain node pair i, j with $x_{ij}^* = 1$. To see the validity of the condition, assume by contradiction

that $y^* \in P(G')$. This implies $y^* = \Sigma_{k=1}^M \lambda_k y^k$, where $y^1, \ldots, y^M$ are characteristic vectors of tours in G', and $\lambda_1, \ldots, \lambda_M$ are nonnegative multipliers with $\Sigma_{k=1}^M \lambda_k = 1$. For each $k \in \{1, \ldots, M\}$ we define x^k as the characteristic vector of the tour of G obtained from y^k by replacing node σ with the arc (i, j). Then, by construction, $x^* = \Sigma_{k=1}^M \lambda_k x^k$, which contradicts the assumption $x^* \notin P(G)$.

It is known that 1-edges cannot be shrunk for STSP. In this respect, ATSP behaves more nicely than STSP, in that the information associated with the orientation of the arcs allows for more powerful shrinkings. We offer the following polyhedral interpretation.

In the separation problem, we are given a point x^* that satisfies the valid inequality $x_{ij} \leq 1$ with equality, and we want to separate it from P. The above discussion shows that this is possible if and only if x^* can be separated from $F := \{x \in P : x_{ij} = 1\}$, hence F can replace P insofar the separation of x^* is concerned. This property is perfectly general, and applies to any nonempty face F of any polytope P. Indeed, let $F := \{x \in P : \beta x = \beta_0\} \neq \emptyset$ be the face of P induced by any valid inequality $\beta x \leq \beta_0$ for P, and assume that $\beta x^* = \beta_0$. If x^* cannot be separated from F, then $x^* \in F \subseteq P$, hence it cannot be separated from P too. Conversely, suppose there exists a valid inequality $\alpha x \leq \alpha_0$ for F that is violated by x^* by the amount $\delta^* := \alpha x^* - \alpha_0 > 0$. Then for a sufficiently large value M, the "lifted" inequality $(\alpha + M\beta)x \leq \alpha_0 + M\beta_0$ is valid for P, but violated by x^* by the amount $(\alpha + M\beta)x^* - (\alpha_0 + M\beta_0) = \alpha x^* - \alpha_0 = \delta^* > 0$.

The different behavior between the asymmetric and symmetric TSP polytopes then has its roots in the basic property that fixing $x_{ij} = 1$ for some ATSP arc (i, j) yields again an ATSP instance—obtained by contracting (i, j) into a single node—whereas the same construction does not work when fixing $y_e = 1$ for some STSP undirected edge e. In polyhedral terms, this means that the face F of the ATSP polytope induced by $x_{ij} \leq 1$ is in 1–1 correspondence with the ATSP polytope on $n - 1$ nodes. Hence, in the asymmetric case, the face F can be interpreted again as an ATSP polytope on a shrunken graph, whereas for STSP a similar interpretation is not possible.

In our implementation, we apply 1-arc shrinking so as to replace each path of 1-arcs by a single node. As a result of this preprocessing on x^*, all the nonzero variables are fractional. Notice that a similar result cannot be obtained for the symmetric TSP, where each 1-edge chain can be replaced by a single 1-edge, but not by a single node.

We have not implemented more sophisticated shrinking procedures in the present version of our code. The above discussion suggests, however, other cases in which a saturated set S can be shrunk. For instance, suppose there exist three distinct nodes i, j, and k such that $x_{ij}^* + x_{ji}^* = 1$ and $x_{jk}^* + x_{kj}^* = 1$, i.e., $x_{ij}^* = x_{jk}^* = \mu$ and $x_{kj}^* = x_{ji}^* = 1 - \mu$ for some $0 \leq \mu \leq 1$. We claim that in this case the saturated set $S = \{i, j\}$ can be shrunk. Indeed, the given point x^* satisfies the valid ATSP inequality $\beta x := x_{ij} + x_{ji} + x_{jk} + x_{kj} \leq \beta_0 := 2$ with equality, hence from the above discussion one can replace P with its face $F := \{x \in P : \beta x = \beta_0\}$. Now, every vertex of F corresponds to a tour using either the path $\{(i, j), (j, k)\}$ or the path $\{(k, j), (j, i)\}$. This property induces a 1-1 correspondence between the vertices of F and those of the ATSP polytope in which i and j have been shrunk into a single node.

3. Pricing with Degeneracy

Pricing is an important ingredient of a branch-and-cut code in that it allows one to effectively handle LP problems involving a huge number of columns. Let

$$z := \min\{cx : Mx \equiv b, x \geq 0\} \tag{13}$$

be the LP problem to be solved. M is an $m \times |A|$ matrix whose columns are indexed by the arcs $(i, j) \in A$. The first $2n - 1$ rows of M correspond to the degree equations (2)–(3) (with the redundant constraint $x(1, V) = 1$ omitted), whereas the remaining rows, if any, correspond to some of the cuts generated through separation. Notation "$\equiv$" stands for "$=$" for the first $2n - 1$ rows of M, and "$\leq$" for the remaining rows. Let M_{ij}^h denote the entry of M indexed by row h and column (i, j).

To keep the size of the LP as small as possible, the following *pricing* scheme is commonly used. We determine a (small) *core* set of arcs, say $\tilde{A}$, and decide to temporarily fix $x_{ij} = 0$ for all $(i, j) \in A \backslash \tilde{A}$. We then solve the restricted LP problem

$$\tilde{z} := \min\{\tilde{c}\tilde{x} : \tilde{M}\tilde{x} \equiv b, \tilde{x} \geq 0\}, \tag{14}$$

where $\tilde{c}$, $\tilde{x}$, and $\tilde{M}$ are obtained from c, x, and M, respectively, by removing all entries indexed by $A \backslash \tilde{A}$.

Assume Problem (14) is feasible and let $\tilde{x}^*$ and $\tilde{u}^*$ be the optimal primal and dual basic solutions found. Clearly, $\tilde{z} \geq z$. We are interested in easily checkable conditions that guarantee $\tilde{z} = z$, thus proving that $\tilde{x}^*$ (with $\tilde{x}^*_{ij} := 0$ for all $(i, j) \in A \backslash \tilde{A}$) is an optimal basic solution to (13), and hence that its value $\tilde{z}$ is a valid lower bound on $v(\text{ATSP})$. To this end, we compute the LP-reduced costs associated with $\tilde{u}^*$, namely

$$\bar{c}_{ij} := c_{ij} - \sum_{h=1}^{m} M^h_{ij} \tilde{u}^*_h \quad \text{for } (i, j) \in A,$$

and check whether $\bar{c}_{ij} \geq 0$ for all $(i, j) \in A$. If this is indeed the case, then $\tilde{z} = z$ and we are done. Otherwise, the current core set $\tilde{A}$ is enlarged by adding (some of) the arcs with negative reduced cost, and the whole procedure is iterated. This iterative solution of (14), followed by the possible updating of $\tilde{A}$, is referred to as the *pricing loop*.

According to our computational experience, the first iterations of the pricing loop tend to add a very large number of new columns to the LP even when $\tilde{z} = z$, due to the typically high primal degeneracy of (14).

As an illustration of this behavior, consider the situation arising when the first LP is solved at the root node of the branching tree. In this case (13) contains the $2n - 1$ degree equations only, hence its optimal solution, x^*, can be computed efficiently through an AP code. Suppose we now initialize the core set $\tilde{A}$ to contain the n arcs chosen in the optimal AP solution. In order to have an LP basis, we add $n - 1$ additional arcs to $\tilde{A}$, chosen so as to determine an $(2n - 1) \times (2n - 1)$ nonsingular matrix $\tilde{M}$. By construction, Problem (14) has a unique feasible solution—namely, the characteristic vector of the optimal AP solution found—hence, we know that $\tilde{z} = z$ holds in this case. However, depending on the possibly "wrong" choice of the last $n - 1$ arcs in the core set, the solution $\tilde{u}^*$ can be dual infeasible for (13), i.e., a (usually very large) number of reduced costs $\bar{c}_{ij}$ are negative. Iterating the pricing procedure produces a similar behavior, and a long sequence of pricings is typically required before all arcs price out correctly.

The above example shows that checking the reduced-cost signs can lead to an over-weak sufficient condition for proving $\tilde{z} = z$. The standard way to cope with this weakness consists of a more careful initialization of the core set, e.g., by taking the 15 smallest cost arcs leaving each node.

We next introduce a new technique, that we call *AP pricing*, in which the pricing condition is strengthened by exploiting the fact that any feasible solution to (13) cannot select the arcs with negative reduced cost in an arbitrary way, as the degree equations—among other constraints—have to be fulfilled.

Let us consider the dual solution $\tilde{u}^*$ to (14) as a vector of Lagrangian multipliers, and the LP reduced costs $\bar{c}_{ij}$ as the corresponding Lagrangian costs. In this view, standard pricing consists of solving the following trivial relaxation of (13):

$$\text{LB}_1 := \min_{x \geq 0} [cx + \tilde{u}^*(b - Mx)] = \tilde{u}^* b + \min_{x \geq 0} \bar{c}x, \quad (15)$$

where $\tilde{u}^* b = \tilde{z}$ by LP duality. Therefore, one has $\tilde{z} + \min_{x \geq 0} \bar{c}x \leq z \leq \tilde{z}$, from which $\tilde{z} = z$ in case $\min_{x \geq 0} \bar{c}x = 0$, i.e., $\bar{c}_{ij} \geq 0$ for all i, j. Our strengthening consists in replacing condition $x \geq 0$ in (15) by

$$x \in F(\text{AP}) := \{x \in \{0, 1\}^A : x(i, V) = x(V, i) = 1 \text{ for all } i \in V\}.$$

In this way, we compute an improved lower bound on z, namely

$$\text{LB}_2 := \tilde{u}^* b + \min_{x \in F(\text{AP})} \bar{c}x = \tilde{z} + \Delta_{\text{AP}},$$

where $\Delta_{\text{AP}} := \min_{x \in F(\text{AP})} \bar{c}x$ is computed efficiently by solving the AP on the Lagrangian costs $\bar{c}_{ij}$. As before, $\tilde{z} + \Delta_{\text{AP}} \leq z \leq \tilde{z}$, hence $\Delta_{\text{AP}} = 0$ implies $\tilde{z} = z$. When $\Delta_{\text{AP}} < 0$, instead, we iterate the procedure, after having added to the core set $\tilde{A}$ the arcs in $A \backslash \tilde{A}$ that are selected in the optimal AP solution found.

The new approach has two main advantages, namely: (i) an improved check for proving $\tilde{z} = z$; and (ii) a better rule to select the arcs to be added to the core arcset. Moreover, LB_2 always gives a lower bound on z (and hence on $v(\text{ATSP})$), which can in some cases succeed in fathoming the current branching node even when $\Delta_{\text{AP}} < 0$. In addition, the nonnegative AP reduced-cost vector $\bar{\bar{c}}$ available after solving $\min_{x \in F(\text{AP})} \bar{c}x$ can be used for fixing $x_{ij} = 0$ for all $(i, j) \in A$ such that $\lceil \text{LB}_2 + \bar{\bar{c}}_{ij} \rceil$ is at least as large as the value of the best known ATSP solution.

We want to mention here that the new pricing scheme can be adapted to other problems having an easily solvable relaxation. For example, in Fischetti and Vigo (1997), the approach is applied to the Resource Constrained Arborescence Problem, the relaxation used for pricing being in this case the Min-Sum Arborescence Problem. Unlike AP, the latter problem involves exponentially many constraints, hence the pricing scheme can in some cases also detect violated cuts that are not present in the current LP, chosen from among those implicitly used during solution of the relaxation. In other words, variable-pricing also produces, as a by-product, a heuristic "pricing" of some exponential classes of cuts, i.e., a separation tool.

AP pricing requires the computation of all reduced costs $\bar{c}_{ij}$. In our implementation, we compute them through the following "row-by-row" scheme. We initialize $\bar{c}_{ij} := c_{ij}$ for all $(i, j) \in A$ and then consider, in sequence, the rows h of M with $\tilde{u}_h^* \neq 0$. For each such row h, we determine the set $A_h := \{(i, j) \in A : M_{ij}^h \neq 0\}$, and update $\bar{c}_{ij} := \bar{c}_{ij} - \tilde{u}_h^* M_{ij}^h$ for $(i, j) \in A_h$. Because of the simple combinatorial structure of the cuts we consider, the (implicit) construction of A_h can be carried out in $O(|A_h|)$ time, hence the overall time spent for computing all reduced costs is $O(n^2)$ for the initialization, plus $O(\sum_{h=1}^m |A_h|)$ for the actual reduced-cost computation (notice that this latter term is linear in the number of nonzero entries in M).

A drawback of AP pricing is the extra computing time spent for the AP solution. After some computational testing, we decided to implement the following strategy. After each LP solution we compute the reduced costs $\bar{c}_{ij}$ and determine the cardinality μ of $A_< := \{(i, j) \in A : \bar{c}_{ij} < 0\}$. If $\mu = 0$, we exit the pricing loop. Otherwise, we check whether μ does not exceed a given parameter, say $\bar{\mu}$, in which case we use standard pricing, i.e., we update $\tilde{A} := \tilde{A} \cup A_<$ and repeat. If $\mu > \bar{\mu}$, instead, we resort to AP pricing, and solve the AP problem on the reduced costs. Clearly, a large value for $\bar{\mu}$ has the drawback of introducing too many variables into the LP; on the other hand, a too small value may lead to an excessive use of the time-consuming AP pricing procedure. We experimentally found that $\bar{\mu} = n/10$ is a good compromise.

When AP pricing is used, we also determine (through a technique to be described later) the arcset Q containing the $2n - 1$ arcs defining an optimal LP basis for this AP problem, and compute the corresponding nonnegative reduced costs $\bar{\bar{c}}_{ij}$ for all $(i, j) \in A$. We remove from Q all the arcs already in $\tilde{A}$, and then enlarge Q by iteratively adding arcs in $(i, j) \in A \backslash (\tilde{A} \cup Q)$ with $\bar{\bar{c}}_{ij} = 0$, until no such arc exists, or $|Q| \geq 50 + n/3$. In this way Q contains a significant number of arcs that are likely to be selected in (13). We finally update $\tilde{A} := \tilde{A} \cup Q$ (even in case $\Delta_{\mathrm{AP}} = 0$), and repeat (if $\Delta_{\mathrm{AP}} < 0$) or exit (if $\Delta_{\mathrm{AP}} = 0$) the pricing loop.

For large-scale problems AP pricing can be excessively time and space consuming. In this case, a possibility is to consider heuristically a sparse set of "candidate arcs" on which the AP problem can be solved more efficiently, and deal with the remaining arcs through the classical pricing scheme. This issue is, however, left to future works: as shown in §5, for all the instances in our test-bed (involving up to 171 nodes) the new approach works quite well.

We conclude this section with an outline of a procedure for computing an optimal LP basis Q associated with the LP problem $\min_{x \in F(\mathrm{AP})} \bar{c}x$, along with the associated reduced costs $\bar{\bar{c}}_{ij}$. Correctness derives from standard network theory. Let A_{AP} contain the n arcs selected in an optimal AP solution, and let $\bar{\bar{c}}_{ij} \geq 0$, $(i, j) \in A$, be the associated AP reduced costs. Both A_{AP} and $\bar{\bar{c}}$ can be computed in $O(n^3)$ time through specialized algorithms (Carpaneto and Toth 1987). We construct a bipartite graph $G_B = (V' \cup V'', E)$ containing two nodes, say $v' \in V'$ and $v'' \in V''$, for each node $v \in V$, and an edge $[i', j'']$ for each arc $(i, j) \in A$. Each edge $[i', j'']$ has cost $\bar{\bar{c}}_{ij}$. We define a fictitious orientation for the edges of G_B, namely, the $n - 1$ edges associated with arcs in $A_{\mathrm{AP}} \backslash \delta^+(1)$ are directed from V'' to V', whereas all other edges have the opposite direction. We then compute a shortest-path arborescence rooted at $1'$, and define Q to contain the $2n - 1$ arcs in A that correspond to the edges selected in the arborescence (with $A_{\mathrm{AP}} \subset Q$). As to reduced costs $\bar{\bar{c}}_{ij}$, they are updated so as to have $\bar{\bar{c}}_{ij} = 0$ for all $(i, j) \in Q$ by setting for all $(i, j) \in A$: $\bar{\bar{c}}_{ij} := 0$ if $(i, j) \in A_{\mathrm{AP}} \backslash \delta^+(1)$, and $\bar{\bar{c}}_{ij} := \bar{\bar{c}}_{ij} + L_{i'} - L_{j''}$ otherwise. Here L_v gives the length of the shortest path from $1'$ to $v \in V' \cup V''$, with $L_{1'} = 0$. This operation corresponds to an updating of the optimal dual solution found by the AP algorithm so as to get an equivalent *basic* solution, obtained by decreasing the dual variables associated with

the outdegree equations $x(i, V) = 1$ (by the amount $L_{i'}$), while increasing those associated with the indegree equations $x(V, j) = 1$ (by the amount $L_{j''}$).

4. The Branch-and-Cut Algorithm

Our branch-and-cut algorithm follows the general framework proposed by Padberg and Rinaldi (1991). We next describe some of its main ingredients.

The algorithm is a lowest-first branch-and-bound procedure, in which lower bounds are computed by means of an LP relaxation that is tightened, at run time, by the addition of facet-defining cuts belonging to some specific classes.

At each node of the branching tree, we initialize the LP relaxation by taking all the constraints present in the last LP solved at the father node (for the root node, only the degree equations are taken). As to variables, we retrieve from a scratch file the optimal basis associated with the last LP solved at the father node, and initialize the core variable set, $\tilde{A}$, by taking all the arcs belonging to this basis (for the root node, $\tilde{A}$ contains the $2n - 1$ variables in the optimal AP basis found by solving AP on the original costs c_{ij}; see §3). In addition, we put into $\tilde{A}$ all the arcs of the best known ATSP solution. Starting with the above advanced basis, we iteratively solve the current LP, apply the AP pricing (and variable fixing) procedure described in §3, and repeat if needed. Observe that the pricing/fixing procedure is applied after each LP solution.

On exit of the pricing loop (case $\Delta_{AP} = 0$), we remove from the current LP the cuts whose associated slack exceeds 0.01 (unless the number of these cuts is less than 10, in which case we keep all the LP rows), and update the LP basis accordingly. Moreover, we apply separation algorithms to find, if any, facet-defining ATSP inequalities that cut off the current LP optimal solution, say x^*. To avoid adding too many cuts to the LP, we skip the violated cuts with degree of violation less than a given positive threshold ϵ, and interrupt the separation phase as soon as we find a sufficient number θ (say) of violated cuts. After computational testing, we decided to set $\epsilon = 0.01$ for SECs, $\epsilon = 0.1$ for the remaining inequalities, and $\theta = 20 + \lfloor n/5 \rfloor$. We first check for violation the cuts generated during the processing of the current or previous nodes, all of which are stored in a global data structure called the *pool*. If some of these cuts are indeed violated by x^*, we exit the separation phase. Otherwise, we apply the Padberg-Rinaldi (1990a) MINCUT algorithm for SEC separation, and exit the separation phase if violated SECs are found. When this is not the case, we shrink the 1-arc paths of x^* (as described in §2.4), and apply the separation algorithms for comb (§2.1), D_k^+ and D_k^- (§2.2), and odd CAT (§2.3) inequalities. To avoid finding equivalent inequalities, we never separate D_3^- inequalities (which are the same as D_3^+ inequalities), and skip odd CAT separation when a violated comb is found (as the class of comb and odd CAT inequalities overlap). When violated cuts are found, we add them to the current LP, and repeat.

When separation fails and x^* is integer, we update the current best ATSP solution, and backtrack. If x^* is fractional, instead, we save the current LP basis in a file, and branch on the variable x_{ij} with $0 < x_{ij}^* < 1$ that maximizes the score $\sigma(i, j) := c_{ij} \cdot \min\{x_{ij}^*, 1 - x_{ij}^*\}$. As a heuristic rule, we give a large priority to the variables with $0.4 \leq x_{ij}^* \leq 0.8$ (if any), so as to produce a significant change in both son nodes.

As a heuristic trailing-off rule, we also branch when the current x^* is fractional and the lower bound did not increase in the last 5 (10 for the root node) LP/pricing/separation iterations.

A simple heuristic algorithm is used to hopefully update the current best optimal ATSP solution. The algorithm is based on the information associated with the current LP, and consists of a complete enumeration of the Hamiltonian circuits in the support graph of x^*, defined as $G^* := (V, \{(i, j) \in A : x_{ij}^* > 0\})$. To this end, the Martello (1983) implicit enumeration algorithm HC is used, with at most $100 + 10n$ backtracking steps allowed. As G^* is typically very sparse, this upper bound on the number of backtrackings is seldom attained, and HC almost always succeeds in completing the enumeration within a short computing time. We apply the heuristic whenever SEC separation fails, since in this case G^* is guaranteed to be strongly connected.

5. Computational Results

The branch-and-cut algorithm of the previous section has been implemented in FORTRAN, and run on a PC 486/33 computer. According to our experience, this

computer runs at approximately 10 MIPS (1.3 MFlops). As we will see, this simple hardware proved to be adequate for the solution of all the real-world test problems from the literature, which involve up to 171 nodes.

We have used Marsten's (1981) XMP code as LP solver; clearly, faster and more robust LP solvers would produce improved performances. AP problems have been solved through the code CTCS described in Carpaneto and Toth (1987), whereas for SEC separation we have used the Padberg-Rinaldi (1990a) original implementation of procedure MINCUT. All these codes are written in FORTRAN language.

Our computational study considers both random and real-world instances. We compare the performances of our branch-and-cut algorithm, called FT-96 in the sequel, with those of the Fischetti-Toth (1992) branch-and-bound code, FT-92. To the best of our knowledge, FT-92 is the most effective branch-and-bound technique for solving to optimality hard ATSP instances. (The other branch-and-bound codes from the literature, such as those in Carpaneto et al. 1995 and Pekny and Miller 1992, make no attempt to improve the AP-bound, hence they are considerably slower on the test problems we consider in this section; see Fischetti and Toth 1992.)

We first considered the following four classes of random instances, taken from Fischetti and Toth (1992):

Class A: c_{ij} uniformly random in range (1–1,000);

Class B: $c_{ij} := \sigma_{ij} + \alpha_{ij}$, with $\sigma_{ij} = \sigma_{ji}$ uniformly random in range (1–1,000), and α_{ij} uniformly random in (1–20);

Class C: $c_{ij} := d_{ij} + \alpha_{ij}$, with $d_{ij} := d_{ji} := \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$, where (x_i, y_i) are uniformly random points in a $1{,}000 \times 1{,}000$ square, and α_{ij}s are as in Class B;

Class D: As in Class C, with points (x_i, y_i) uniformly generated inside five "clusters" (each cluster is represented by a square whose area is 20,000 and whose center is a uniformly random point inside the $1{,}000 \times 1{,}000$ square).

All costs have been truncated so as to obtain integer values.

Instances of Class A are typically very easy for all AP-based codes, hence they do not belong to the class of "hard" instances we aim to address in this paper. We decided anyway to report some statistics for this class too, mainly to quantify the reduction in the number of branching nodes resulting from the use of additional inequalities.

The instances of Classes B, C, and D are instead much harder for the AP-based codes, as the costs are almost symmetric (i.e., $c_{ij} \approx c_{ji}$ for all i, j). Problems of Class D represent real-life situations in which the cities to be visited are clustered in a few geographical regions.

For each problem class and each value of n, nine instances have been generated and solved by FT-92 and FT-96. Results for FT-92 are taken from Repetto (1994), who ran the algorithm on a DEC 3000-400 Alpha AXP (133 Mhz) computer with a time limit of 3,600 CPU seconds. According to our experience, DEC Alpha is about 10 times as fast as PC 486/33. As to FT-96, we set the time limit to 5,000 seconds—instead of 36,000 seconds—of elapsed time on the PC 486/33 computer. For both algorithms, statistics refer to the solved instances only.

Table 1 reports the average results obtained for Classes A to D, respectively. The real-world instances considered in Fischetti and Toth (1992) are addressed in Table 2. The columns of the tables give:

n: Number of nodes.

Time: Computing time, in PC 486/33 seconds; for FT-92, the times reported in Repetto (1994) have been multiplied by 10 to take into account the difference in computing speed. Time limit is 36,000 PC-equivalent seconds for FT-92, and 5,000 PC seconds for FT-96. The number of nonsolved instances (if different from 0) is reported in parenthesis.

Nod: Number of nodes generated in the branching tree.

%-Gap: Percentage gap between the root-node lower bound and the optimal ATSP value.

%-SEC: Percentage gap between the optimal value of the LP relaxation (1)–(5) and the optimal ATSP value.

SEC: Number of SECs generated by FT-96.

COMB: Number of comb inequalities generated by FT-96.

DK: Number of (clique-lifted) D_k^+ and D_k^- inequalities generated by FT-96.

CAT: Number of (clique-lifted) odd CAT inequalities generated by FT-96.

Cuts: Total number of cuts generated by FT-96 (observe that SEC + COMB + DK + CAT can be smaller

Table 1 Results from Four Classes of Random Instances

	FT-92			FT-96								
n	Time	Nod	%-Gap	Time	Nod	%-Gap	%-SEC	SEC	COMB	DK	CAT	Cuts
					Class A Statistics							
20	0.1	18	3.4	2.2	2	0.34	0.89	2	0	7	2	12
30	0.2	25	3.0	7.7	4	0.58	1.01	7	0	14	16	38
40	0.6	35	2.3	8.6	4	0.15	0.38	6	0	9	11	27
50	0.8	38	0.6	10.9	4	0.17	0.32	6	0	8	13	29
60	1.3	55	1.1	19.6	7	0.18	0.28	10	0	13	18	43
80	5.0	89	1.0	53.1	12	0.22	0.31	13	0	25	34	73
100	4.8	76	0.7	17.7	3	0.06	0.22	8	0	8	13	30
					Class B Statistics							
20	0.4	45	3.2	3.3	2	0.08	0.31	13	0	4	9	28
30	2.9	144	6.0	9.1	3	0.24	0.43	18	0	9	22	51
40	8.7	223	5.1	39.4	9	0.46	0.87	30	1	19	73	124
50	117.8	1,736	6.4	23.2	5	0.23	0.43	30	0	7	50	88
60	87.4	857	4.3	136.3	19	0.48	0.75	48	1	23	167	241
80	1,219.6	5,700	5.1	192.3	17	0.29	0.48	59	1	27	151	239
100	4,216(1)	12,812	4.2	337.9	21	0.25	0.42	72	1	24	216	315
					Class C Statistics							
20	0.8	62	2.0	2.8	1	0.00	0.24	18	0	1	0	20
30	29.0	1,209	2.8	5.7	1	0.00	0.19	23	1	6	9	41
40	502.6	10,533	3.3	83.7	9	0.36	1.20	41	10	35	56	144
50	4,632(1)	12,812	4.2	110.8	12	0.23	0.77	51	10	31	46	139
60		not tried		223.3	16	0.37	1.15	70	17	45	74	207
80		not tried		633.6	30	0.40	1.01	104	19	68	124	317
100		not tried		1,826(3)	51	0.51	1.93	179	30	104	227	541
					Class D Statistics							
20	3.0	192	0.9	3.2	1	0.00	0.10	23	0	2	5	31
30	581.7	27,257	1.5	8.6	2	0.04	0.08	36	0	6	11	55
40	2,060(1)	69,944	1.5	27.4	3	0.09	0.43	51	2	21	28	103
50	7,240(3)	77,102	1.7	39.2	3	0.10	0.51	61	4	30	32	128
60		not tried		51.2	4	0.09	0.29	68	1	24	32	127
80		not tried		323.5	16	0.24	0.54	97	7	50	67	222
100		not tried		1,303.2	23	0.31	0.77	137	12	82	128	360

than Cuts, since all these values have been truncated to integers).

As expected, FT-96 is worse than FT-92 on the easy instances of Class A, although it requires much fewer branching nodes. For the remaining classes, FT-96 clearly outperforms FT-92. These results confirm the common folklore that, on hard instances, an AP-based branch-and-bound code is inferior to a branch-and-cut algorithm.

On the whole, the performance of FT-96 on all classes is rather satisfactory, although we believe that for almost symmetric instances a substantial improvement can be obtained by exploiting other separation procedures from the STSP literature. Since in this paper we are mainly concerned with the design of new separation tools for asymmetric inequalities, we leave this implementation issue for future research.

Table 2 Some Real-World Instances

		FT-92			FT-96								
Name	*n*	Time	Nod	%-Gap	Time	Nod	%-Gap	%-SEC	SEC	COMB	DK	CAT	Cuts
FT53	53	7.6	147	1.6	6.1	1	0.00	0.00	20	0	0	0	20
FT70	70	19.2	276	0.6	23.7	5	0.02	0.05	25	0	16	17	58
RY48P	48	729.6	14,050	2.9	52.8	5	0.53	0.92	58	6	42	35	141
KRO124P	100	6,479.0	15,842	2.7	79.0	3	0.04	0.63	80	5	53	18	156

Table 3 reports the performance of our branch-and-cut code on some additional real-world ATSP instances, which are very difficult to solve by using an AP-based code. Problem P43 is a scheduling problem arising in chemical engineering (Balas et al. 1993). Problems FTV33 to FTV170 are vehicle routing problems, and refer to pharmaceutical product delivery within downtown Bologna. Problems FTV33 to FTV70 are from Fischetti et al. (1994), whereas FTV170 is new. As to FTV90, FTV100, FTV110, FTV120, FTV130, FTV140, FTV150, and FTV160, they are obtained from FTV170 by taking the first 91, 101, 111, 121, 131, 141, 151, and 161 nodes, respectively. All problems are now part of the Reinelt's TSPLIB library (1991); they are also available, on request, from the authors.

The columns of Table 3 give:

OptVal: Optimal ATSP value;

t-LP: Time spent within the LP solver;

t-PRI: Time spend within the pricing/fixing procedure;

t-SEP: Overall time spent for separation;

the remaining columns are as in Table 2.

Table 3 Real-World Instances

Name	*n*	OptVal	Time	*t*-LP	*t*-PRI	*t*-SEP	Nod	%-Gap	SEC	COMB	DK	CAT	Cuts
FT53	53	6,905	6.1	3.2	1.6	0.1	1	0.00	20	0	0	0	20
FT70	70	38,673	23.7	8.5	7.3	3.3	5	0.02	25	0	16	17	58
RY48P	48	14,422	52.8	37.5	3.8	6.3	5	0.53	58	6	42	35	141
KRO124P	100	36,230	79.0	55.5	10.2	6.9	3	0.04	80	5	53	18	156
P43	43	5,620	492.2	357.0	27.7	46.2	73	0.16	76	17	189	175	457
FTV33	34	1,286	2.7	0.8	0.5	0.1	1	0.00	20	0	0	0	20
FTV35	36	1,473	18.9	11.3	1.3	2.6	7	0.88	34	0	36	4	74
FTV38	39	1,530	42.3	22.1	3.8	5.8	17	0.85	34	0	56	13	103
FTV44	45	1,613	34.1	16.2	2.9	7.5	7	0.37	31	0	40	55	126
FTV47	48	1,776	70.3	35.0	6.6	11.1	27	1.01	38	0	55	66	159
FTV55	56	1,608	169.1	94.0	18.9	21.8	45	0.81	49	4	113	49	215
FTV64	65	1,839	165.9	92.9	22.8	24.5	25	1.25	46	10	114	48	218
FTV70	71	1,950	137.4	84.4	16.6	10.9	31	0.92	41	6	69	14	130
FTV90	91	1,579	44.5	18.5	7.1	7.3	9	0.25	29	8	29	64	130
FTV100	101	1,788	50.0	21.7	9.1	8.6	5	0.22	35	12	49	50	146
FTV110	111	1,958	431.2	205.3	69.3	85.3	45	0.72	44	10	109	117	280
FTV120	121	2,166	1,704.5	1,087.9	316.9	102.6	153	1.06	52	8	100	96	256
FTV130	131	2,307	231.2	99.1	42.0	45.8	28	0.35	44	8	54	95	201
FTV140	141	2,420	225.1	101.2	48.2	32.4	29	0.25	45	10	49	61	165
FTV150	151	2,611	338.6	166.5	68.9	44.0	39	0.27	48	10	61	72	191
FTV160	161	2,683	1,076.4	568.9	261.3	109.3	73	0.67	66	30	213	101	410
FTV170	171	2,755	434.6	241.5	92.6	59.4	17	0.80	64	34	84	55	237

Our branch-and-cut code spent about 60% of the overall computing time within the LP solver; hence the use of an LP solver faster than XMP would result in a significant increase in speed.

As for separation, the most time-consuming procedures are those for detecting violated SECs and for searching the constraint pool, whereas both procedures for D_k and odd CAT separation are quite fast. For example, for problems FTV150, FTV160, and FTV170, respectively, the algorithm spent 7.8, 51.7, and 13.8 seconds for searching the pool; 17.1, 37.3, and 27.4 seconds for SEC separation; 6.2, 4.9, and 7.0 seconds for D_k separation; and 9.7, 10.5, and 7.3 for odd CAT and comb separation.

The time spent within the pricing/fixing procedure amounts to about 20% of the overall time. Notice that this procedure is called after each LP solution, hence our implementation seems to be rather efficient—at least for the problem sizes we consider. According to our experience, no improvement of the overall performances is obtained if the pricing/fixing procedure is called less frequently (e.g., calling it at every fifth LP iteration increases the overall average computing time of about 15%).

To evaluate the effectiveness of AP pricing in dealing with LP degeneracy, we implemented a modified version of our branch-and-cut code in which the classical pricing scheme is used. At the root node of the branching tree, the core arc set $\tilde{A}$ is initialized by taking the arcs of the optimal AP solution, plus the θ (say) min-cost arcs leaving each node (the original arc costs are used). After each kth LP solution, we apply the standard column-by-column pricing scheme. As in Padberg and Rinaldi (1991), we use a "truncated" pricing scheme, i.e., for each variable we stop the reduced-cost computation as soon as we are sure the final reduced cost cannot be negative. However, at every qth pricing call in which all variables price out correctly, we compute each reduced cost exactly in order to apply variable-fixing criteria. Clearly, pricing and fixing are also invoked

Table 4 Pricing Strategies

	Classical Pricing					AP Pricing				
Name	Time	Nod	*t*-PRI	*t*-LP	*n* Core	Time	Nod	*t*-PRI	*t*-LP	*n* Core
FT53	9.9	1	0.7	6.0	615	6.1	1	1.6	3.2	235
FT70	50.3	5	3.6	33.2	973	23.7	5	7.3	8.5	256
RY48P	63.6	5	1.0	45.4	490	52.8	5	3.8	37.5	266
KRO124P	67.9	3	7.5	41.7	977	79.0	3	10.2	55.5	505
P43	3,056.4	89	82.5	2,665.5	517	492.2	73	27.7	357.0	623
FTV33	3.9	1	0.0	1.8	399	2.7	1	0.5	0.8	160
FTV35	25.4	7	1.0	12.2	508	18.9	7	1.3	11.3	161
FTV38	46.8	17	0.7	19.6	356	42.3	17	3.8	22.1	168
FTV44	118.2	17	2.1	69.7	436	34.1	7	2.9	16.2	238
FTV47	58.2	15	2.5	28.5	502	70.3	27	6.6	35.0	259
FTV55	342.6	53	10.8	241.7	565	169.1	45	18.9	94.0	295
FTV64	161.8	15	5.0	116.5	667	165.9	25	22.8	92.9	347
FTV70	395.3	43	14.3	254.9	766	137.4	31	16.6	84.4	404
FTV90	64.7	11	5.0	28.6	919	44.5	9	7.1	18.5	420
FTV100	136.0	11	16.4	62.5	766	50.0	5	9.1	21.7	548
FTV110	500.8	41	38.3	295.7	763	431.2	45	69.3	205.3	609
FTV120	2,039.6	127	145.7	1,465.1	823	1,704.5	153	316.9	1,087.9	646
FTV130	385.0	31	75.1	149.2	916	231.2	28	42.0	99.1	696
FTV140	275.7	11	69.3	102.8	944	225.1	29	48.2	101.2	718
FTV150	285.0	16	56.5	124.7	1,063	338.6	39	68.9	166.5	734
FTV160	3,766.0	149	196.3	2,567.7	1,118	1,076.4	73	261.3	568.9	851
FTV170	433.5	19	34.0	256.4	1,220	434.6	17	92.6	241.5	914

Table 5 Modified Branch-and-Cut Codes

	SEC Only				SEC + DK					SEC + COMB + CAT					
Name	Time	Nod	%-Gap	SEC	Time	Nod	%-Gap	SEC	DK	Time	Nod	%-Gap	SEC	COMB	CAT
FT53	6.3	1	0.00	20	5.7	1	0.00	20	0	5.5	1	0.00	20	0	0
FT70	19.7	7	0.05	25	22.2	7	0.03	25	23	16.4	5	0.02	25	0	17
RY48P	159.0	77	0.92	60	111.2	21	0.75	64	59	64.7	9	0.69	55	8	67
KRO124P	350.3	53	0.63	94	118.4	9	0.29	72	42	55.2	3	0.10	72	6	31
P43	517.7	167	0.16	84	744.5	101	0.16	146	266	498.3	105	0.16	89	23	299
FTV33	2.7	1	0.00	20	1.8	1	0.00	20	0	1.9	1	0.00	20	0	0
FTV35	19.9	13	1.02	41	14.5	7	0.88	32	34	13.4	11	1.02	39	0	12
FTV38	49.4	27	0.98	60	50.3	23	0.85	40	73	15.2	17	0.98	36	0	4
FTV44	86.1	45	1.74	60	26.3	9	1.24	32	46	29.8	9	0.43	45	2	60
FTV47	169.1	113	1.52	51	40.8	23	1.01	35	46	70.6	43	1.24	42	2	52
FTV55	309.1	215	1.49	66	448.1	125	1.43	61	281	141.7	77	1.00	43	4	44
FTV64	179.8	99	1.69	48	698.4	117	1.63	74	285	152.8	31	1.14	52	10	72
FTV70	325.7	173	2.10	71	441.8	93	1.69	76	159	119.9	53	1.03	33	6	40
FTV90	90.1	47	0.57	35	82.5	11	1.57	31	60	45.8	11	0.25	26	8	78
FTV100	206.4	91	1.12	46	377.6	67	0.73	47	212	113.3	23	0.28	39	10	77
FTV110	536.7	193	1.74	48	556.8	115	1.48	42	107	191.0	33	0.72	37	10	122
FTV120	1,643.9	449	1.62	69	>10,000	775	1.62	135	686	1,482.9	231	1.06	73	12	247
FTV130	1,872.0	557	0.87	64	1,233.5	151	0.65	65	203	318.1	47	0.35	60	8	110
FTV140	618.6	201	0.83	55	606.8	81	0.62	50	139	559.2	67	0.29	73	12	147
FTV150	616.6	183	0.77	65	>10,000	360	0.58	130	577	358.3	49	0.27	56	13	104
FTV160	>10,000	1,092	1.34	173	>10,000	608	1.05	194	613	864.5	99	0.71	73	24	168
FTV170	>10,000	901	1.45	200	>10,000	641	1.20	128	328	>10,000	443	0.93	188	36	255

whenever this is required for the branch-and-cut correctness, e.g., just before branching. As in the AP-pricing scheme, at each round of pricing we never add more than $50 + n/3$ variables to the current core.

An extensive computational testing on the real-world instances of our test bed showed that values $\theta = 5$, $k = 10$, and $q = 1$ lead to the best average results. In particular, we found that larger values for θ produce significantly larger LPs and hence consistently worse results (e.g., setting $\theta = 10$ yields to an average increase of the overall computing time of about 70%). Changing the pricing and fixing parameters k and q has instead a smaller effect on the algorithm performance (e.g., setting $k = 1$ and $k = 15$ produces an average computing time increase of 10% and 5%, respectively).

Table 4 compares the classical (best tuned) and new pricing schemes on the real-world instances of Table 3. The table column *n-core* gives the maximum core size (number of LP variables); the remaining columns are as in Table 3.

According to Table 4, the classical pricing scheme typically produces significantly larger and/or more degenerate LPs; as a result, the overall computing time increases considerably. Clearly, different results could have been obtained by considering a different LP-solver or a more sophisticated implementation of the classical pricing scheme. Moreover, as already observed, AP pricing (as stated) can be very time and space consuming on large-scale instances. Hence, we cannot claim the new approach always outperforms any implementation of the standard pricing idea. We believe, however, that the results show the effectiveness of the AP-pricing scheme in dealing with highly degenerate LP problems, at least for ATSP problems with a few hundred nodes.

Additional results are reported in Table 5, where we investigate the performances of a modified branch-and-cut code obtained by inhibiting the use of some of the proposed separation procedures. We considered three cases: (i) SEC separation only, (ii) SEC and D_k separation only, and (iii) SEC, comb and odd CAT separation

only. The time limit for these modified codes has been set to 10,000 PC-486/33 seconds.

According to the table, some of the instances—including the problem P43 mentioned in the introduction—can be solved rather easily even by considering only SECs, i.e., these instances are not "hard" if a branch-and-cut code is used instead of an AP-based branch-and-bound algorithm. This confirms the findings of Balas et al. (1993) and of Jünger et al. (1995).

On the other hand, some instances of our test bed cannot be solved (within the 10,000 second time limit) if additional inequalities are not exploited; in particular, these new inequalities turn out to be of crucial importance for the solution of FTV160 and FTV170.

The algorithm using SECs and D_k inequalities exhibits, a worse performance than the one using only SECs: When using D_k inequalities, the average time spent to process each node of the branching tree almost doubles, whereas the total number of these nodes does not reduce considerably. For some instances, the number of branching nodes even increases if D_k inequalities are used. An explanation is that the new cuts in some cases produce "wild" fractional points leading to worse choices of the branching variables.

Much better performances are obtained if comb and odd CAT inequalities are used instead of D_k inequalities. In particular, our separation scheme produces a large number of violated (clique-lifted) odd CAT inequalities, leading to a substantial reduction of the number of branching nodes. As to comb inequalities, they do not seem to play an important role for these instances: We also ran the code without comb separation, with more or less the same results—this could, however, be due to the very simple nature of our heuristic for comb separation. In some cases, better results are obtained if D_k inequalities are combined with (comb and) odd CAT inequalities, i.e., if all the proposed separations are applied; cfr. Table 3. In particular, the largest instance of our test bed, FTV170, cannot be solved within the 10,000-second time limit by any of the modified algorithms of Table 5.

6. Conclusions

We have described a branch-and-cut algorithm for the exact solution of the asymmetric traveling salesman problem (ATSP). To our knowledge, this is the first polyhedral method proposed in the literature for an "ad hoc" solution of ATSP instances.

The algorithm uses new separation procedures for large classes of facet-defining inequalities, namely (clique-lifted) D_k^+, D_k^-, and odd CAT inequalities.

We have discussed and analyzed computationally a new technique for variable pricing and fixing, which has some advantages with respect to the standard one—at least for problems involving a few hundred nodes. We have also addressed sufficient conditions for node shrinking in the context of separation.

Computational results are presented, showing that the proposed algorithm outperforms the best branch-and-bound algorithm from the literature when "hard" instances are considered. In particular, we were able to solve to optimality, on a PC 486/33, all the real-world ATSP test problems in Reinelt's TSPLIB library.

We have also investigated the relative strength of SEC, D_k, and odd CAT inequalities for real-world test problem involving up to 171 nodes. For the hardest instances of our test bed, the (clique-lifted) odd CAT inequalities proved of crucial importance for an effective solution of the problem.

Future research on the subject should be devoted to the definition of improved separation procedures. In particular, more general lifted cycle inequalities should be addressed. Indeed, for several instances that cannot be solved at the root node by the present code (including problem P43), we were able to detect "by hand" several violated lifted cycle inequalities not belonging to the D_k^+ and D_k^- classes. Also, we are confident that more powerful shrinking procedures can improve the overall algorithm significantly.[1]

[1] This research has been supported by MURST and CNR, Italy. Some of the results of §2.2 follow from a joint research project of the first author with Norbert Ascheuer and Martin Grötschel on ATSP with side constraints. We thank Manfred Padberg and Giovanni Rinaldi for having kindly provided us with their FORTRAN code MINCUT for SEC separation.

References

Applegate, D., R. E. Bixby, V. Chvátal, and W. Cook, "Finding Cuts in the TSP (*a preliminary report*)," Technical Report 95-05, DIMACS, Rutgers Univ., New Brunswick, NJ, 1995.

Balas, E., "The Asymmetric Assignment Problem and Some New Facets of the Traveling Salesman Polytope on a Directed Graph," *SIAM J. Discrete Math.*, 2 (1989), 425–451.

Balas, E., S. Ceria, and G. Cornuéjols, "A Lift-and-Project Cutting Plane Algorithm for Mixed 0-1 Programs," *Math. Programming*, 58 (1993), 295–324.

—— and M. Fischetti, "The Fixed Out-Degree 1-Arborescence Polytope," *Math. Oper. Res.*, 17 (1992), 1001–1018.

—— and ——, "A Lifting Procedure for the Asymmetric Traveling Salesman Polytope and a Large New Class of Facets," *Math. Programming*, 58 (1993a), 325–352.

—— and ——, "On the Monotonization of Polyhedra," in G. Rinaldi and L. A. Wolsey (Eds.), *Integer Programming and Combinatorial Optimization*, IPCO III Proceedings, Erice, Italy, 1993b, 23–38.

—— and P. Toth, "Branch and Bound Methods," in E. L. Lawler, J. K. Lenstra, A. H. G. Rinnooy Kan and D. B. Shmoys (Eds.), *The Traveling Salesman Problem*, John Wiley and Sons, New York, 1985.

Caprara, A. and M. Fischetti, "0-1/2 Chvátal-Gomory cuts," *Math. Programming*, 74 (1996), 221–235.

Carpaneto, G., M. Dell'Amico, and P. Toth, "Exact Solution of Large-Scale, Asymmetric Traveling Salesman Problems," *ACM Trans. on Math. Software*, 21 (1995), 394–409.

—— and P. Toth, "Primal-Dual Algorithms for the Assignment Problem," *Discrete Applied Math.*, 18 (1987), 137–153.

Chopra, S. and G. Rinaldi, "The Graphical Asymmetric Travelling Salesman Polyhedron," in R. Kannan and W. R. Pulleyblank (Eds.), IPCO I Proceedings, University of Waterloo, Ontario, Canada, 1990.

Clochard, J. M. and D. Naddef, "Using Path Inequalities in a Branch-and-Cut Code for the Symmetric Traveling Salesman Problem," in G. Rinaldi and L. A. Wolsey (Eds.), *Integer Programming and Combinatorial Optimization*, IPCO III Proceedings, Erice, Italy, 1993, 291–311.

Fischetti, M., "Facets of the Asymmetric Traveling Salesman Polytope," *Math. Oper. Res.*, 16 (1991), 42–56.

——, "Three Lifting Theorems for the Asymmetric Traveling Salesman Polytope," in E. Balas, G. Cornuéjols, and R. Kannan (Eds.), *Integer Programming and Combinatorial Optimization*, IPCO II Proceedings, Carnegie Mellon University, Pittsburgh, PA, 1992, 260–273.

——, "Clique Tree Inequalities Define Facets of the Asymmetric Traveling Salesman Polytope," *Discrete Applied Math.*, 56 (1995), 9–18.

—— and P. Toth, "An Additive Bounding Procedure for the Asymmetric Travelling Salesman Problem," *Math. Programming*, 53 (1992), 173–197.

——, ——, and D. Vigo, "A Branch and Bound Algorithm for the Capacitated Vehicle Routing Problem on Directed Graphs," *Oper. Res.*, 42 (1994), 846–859.

—— and D. Vigo, "A Branch and Cut Algorithm for the Resource-Constrained Min-Sum Arborescence Problem," *Networks*, 29 (1997), 55–67.

Grötschel, M. and M. W. Padberg, "Polyhedral Theory," in E. L. Lawler, J. K. Lenstra, A. H. G. Rinnooy Kan and D. B. Shmoys (Eds.), *The Traveling Salesman Problem*, John Wiley and Sons, New York, 1985.

Gerards, A. M. H. and A. Schrijver, "Matrices with the Edmonds-Johnson Property," *Combinatorica*, 6 (1986), 365–379.

Jünger, M., G. Reinelt, and G. Rinaldi, "The Traveling Salesman Problem," in M. Ball, T. L. Magnanti, C. L. Monma, and G. Nemhauser (Eds.), *Handbooks in Operations Research and Management Science*, Vol. 7, "Network Models," North-Holland, Amsterdam, The Netherlands, 1995, 255–330.

Laporte, G., "The Traveling Salesman Problem: An Overview of Exact and Approximate Algorithms," *European J. Oper. Res.*, 59 (1992), 231–247.

Marsten, R., "The Design of the XMP Linear Programming Library," *ACM Trans. Math. Software*, 7 (1981), 481–497.

Martello, S., "An Enumerative Algorithm for Finding Hamiltonian Circuits in a Directed Graph," *ACM Trans. on Math. Software*, 9 (1983), 131–138.

Padberg, M. W. and G. Rinaldi, "An Efficient Algorithm for the Minimum Capacity Cut Problem," *Math. Programming*, 47 (1990a), 19–36.

—— and ——, "Facet Identification for the Symmetric Traveling Salesman Polytope," *Math. Programming*, 47 (1990b), 219–257.

—— and ——, "A Branch-and-Cut Algorithm for the Resolution of Large-Scale Symmetric Traveling Salesman Problems," *SIAM Rev.*, 33 (1991), 60–100.

Pekny, J. F. and D. L. Miller, "A Parallel Branch-and-Bound Algorithm for Solving Large Asymmetric Traveling Salesman Problems," *Math. Programming*, 55 (1992), 17–33.

Reinelt, G. "TSPLIB—A Traveling Salesman Problem Library," *ORSA J. Computing*, 3 (1991), 376–384.

Repetto, B., "A Repetitive Application of a Well Known Lower Bounding Procedure for the ATSP," Technical Report, Carnegie Mellon University, Pittsburgh, PA, 1994.

Queyranne, M. and Y. Wang, "Symmetric Inequalities and Their Composition for Asymmetric Travelling Salesman Polytopes," *Math. Oper. Res.*, 20 (1995), 838–863.

Accepted by Thomas M. Liebling; received February 1996. *This paper has been with the authors* 1 *month for* 1 *revision.*